

SUDOKU SOLVER REPORT

1. Introduction

- a. Sudoku Solver is an application designed to solve standard 9x9 Sudoku puzzles. Empty cells are represented by the number **0**. The application uses the backtracking algorithm to search for a valid solution.

- b. Running the program

The project files must be in one folder. To run the program run command “python main.py”

- c. **How to Use the Program**

- 1) Open *input.txt*
- 2) Enter Sudoku puzzle using 9 rows and 9 numbers in each row
- 3) Use 0 for empty cells
- 4) Save the file
- 5) Run the program
- 6) View result in console and *output.txt*

2. Analysis

- 1) `main.py` – starts the program
- 2) `board.py` – Sudoku board class
- 3) `solver.py` – abstract solver class
- 4) `solver_tracker.py` – backtracking solver implementation
- 5) `file_manager.py` – reading/writing files
- 6) `tests/test_solver.py` – unit tests

2.1. OOP pillars

a. Encapsulation

Encapsulation means protecting internal data inside a class.

Example:

```
class SudokuBoard:  
  
    def __init__(self, grid):  
  
        self._grid = grid
```

The `_grid` attribute is stored inside the object and accessed through methods.

b. Abstraction

Abstraction means showing only important functionality.

Example:

```
from abc import ABC, abstractmethod  
  
  
class SudokuSolver(ABC):  
  
    @abstractmethod  
  
    def solve(self, board):  
  
        pass
```

The user knows every solver must have `solve()` method.

c. Inheritance

Inheritance allows one class to reuse another class design.

Example:

```
class BacktrackingSolver(SudokuSolver):
```

BacktrackingSolver inherits from SudokuSolver.

d. **Polymorphism**

Polymorphism means different objects can use the same method name differently.

Example:

```
solver.solve(board)
```

Different solver classes could implement solve() in different ways.

f. **Design Pattern – Singleton**

The project uses the **Singleton** design pattern in the FileManager class. This ensures that only one instance of the file management object exists during program execution.

Example:

```
file_manager = FileManager()
```

If another object is created later, the same instance is returned.

3.1. Results

- A functional Sudoku Solver application was successfully created in Python, capable of reading puzzles from a TXT file, solving them, and saving the results.
- The project correctly implemented the main OOP principles and used the Singleton design pattern.
- One challenge during development was handling invalid input files, such as missing rows or incorrect numbers, so validation checks were added to ensure the file always contains a 9x9 puzzle before solving.
- Another difficulty was debugging the backtracking algorithm when some numbers were placed incorrectly. This was fixed by correcting the 3x3 box validation logic to properly calculate the starting row and column of each subgrid before checking for duplicate values.

3.2. Summary

This coursework achieved the goal of creating an object-oriented Python application. The Sudoku Solver can automatically solve standard 9x9 puzzles using backtracking algorithm. The project demonstrates inheritance, encapsulation, abstraction, and polymorphism in a practical example.

The result is a working console application that uses file input/output and testing. In future versions, the project could be extended with graphical user interface, multiple solving strategies, puzzle generator, difficulty levels, and performance statistics.